

devtown

Python Cheatsheet

Basics

Importing packages

Python packages are a collection of useful tools developed by the open-source community. They extend the capabilities of the python language. To install a new package (for example, pandas), you can go to your command prompt and type in pip install pandas. Once a package is installed, you can import it as follows.

`import pandas` # Import a package without an alias

`import pandas as pd` # Import a package with an alias

`from pandas import dataframe` # Import an object from a package

The working directory

The working directory is the default file path that python reads or saves files into. An example of the working directory is C://file/path . The os library is needed to set and get the working directory.

`import os` # Import the operating system package

`os.getcwd()` # Get the current directory

`os.setcwd("new/working/directory")` # Set the working directory to a new file path

Math functions and methods

All functions take an array as the input

`np.log(x)` # Calculate logarithm
`np.exp(x)` # Calculate exponential
`np.max(x)` # Get maximum value
`np.min(x)` # Get minimum value
`np.sum(x)` # Calculate sum
`np.mean(x)` # Calculate mean

`np.quantile(x, q)` # Calculate q-th quantile
`np.round(x, n)` # Round to n decimal places
`np.std(x)` # Calculate standard deviation
`np.var(x)` # Calculate variance

Operators

Arithmetic operators

`102 + 37` # Add two numbers with +

`102 - 37` # Subtract a number with-

`4 * 6` # Multiply two numbers with *

`22 / 7` # Divide a number by another with/

`22 // 7` # Integer divide a number with //

`3 ^ 4` # Raise to the power with ^

`22 % 7` # Returns 1 # Get the remainder after division with %

Assignment operators

`a = 5` # Assign a value to a

`x[0] = 1` # Change the value of an item in a list

Numeric comparison operators

`3 == 3` # Test for equality with ==

`3 != 3` # Test for inequality with !=

`3 > 1` # Test greater than with >

`3 >= 1` # Test greater than or equal to with >=

`3 < 4` # Test less than with <

`3 <= 4` # Test less than or equal to with <=

Logical operators

`~(2 == 2)` # Logical NOT with ~

`(1 != 1) & (1 < 1)` # Logical AND with &

`(1 >= 1) | (1 < 1)` # Logical OR with |

`(1 != 1) ^ (1 < 1)` # Logical XOR with ^

Getting started with lists

A list is an ordered and changeable sequence of elements. It can hold integers, characters, floats, strings, and even objects.

Creating lists

Create Lists with [], elements separated by commas
`x = [1, 3, 2]`

List functions and methods

`x.sorted(x)` # Return a sorted copy of the list e.g., [1,2,3]
`x.sort()` # Sorts the list in-place (replaces x)
`reversed(x)` # Reverse the order of elements in x e.g., [2,3,1]
`x.reversed()` # Reverse the list in-place
`x.count(2)` # Count the number of element 2 in the list

Selecting list elements

Python lists are zero-indexed (the first element has index 0). For ranges, the first element is included but the last is not.

`x= ['a', 'b', 'c', 'd', 'e']` # Define the list
`x[0]` # Select the 0th element in the list
`x[-1]` # Select the last element in the list
`x[1:3]` # Select 1st (inclusive) to 3rd (exclusive)
`x[2]` # Select the 2nd to the end
`x[:3]` # Select 0th to 3rd (exclusive)

Concatenating lists

Define the x and y lists
`x = [1, 3, 6]`
`y = [10, 15, 21]`

`x + y` # Returns [1, 3, 6, 10, 15, 21]
`3x` # Returns [1, 3, 6, 1, 3, 6, 1, 3, 6]

Getting started with characters and strings

Create a string with double or single quotes
`"DataCamp"`

Embed a quote in string with the escape character \
`"He said, \"DataCamp\""`

Create multi-line strings with triple quotes
`"""`
`A Frame of Data`
`Tidy, Mine, Analyze It`
`Now You Have Meaning`
`x.sort()` # Sorts the list in-place (replaces x)
`Citation: https://mdsr-book.github.io/haikus.html`

`str[0]` # Get the character at a specific position
`str[0:2]` # Get a substring from starting to ending index (exclusive)

Combining and splitting strings

`"Data" + "Framed"` # Concatenate strings with, this returns 'DataFramed'
`3 "data"` # Repeat strings with *, this returns 'data data data'
`"beekeepers".split("e")` # Split a string on a delimiter, returns ['b', ' ', 'p', 'rs']

Mutate strings

`str = "Jack and Jill"` # Define str
`str.upper()` # Convert a string to uppercase, returns 'JACK AND JILL'
`str.lower()` # Convert a string to lowercase, returns 'jack and jill'
`str.title()` # Convert a string to title case, returns 'Jack And Jill'
`str.replace("J", "P")` # Replaces matches of a substring with another, returns 'Pack and Pill'